

Snowflake Lab: ETL

Goal: Load a CSV file into a Snowflake table

Step-by-Step Instructions

1. Prepare the CSV file

Open Notepad → paste the text below → save the file as employees.csv on your Desktop.

```
EMPLOYEE_ID,FIRST_NAME,DEPARTMENT,SALARY,START_DATE
101,Aisha,Marketing,75000,2025-01-10
102,Ravi,Engineering,95000,2025-02-15
103,Sofia,HR,68000,2025-03-01
104,Li,Finance,82000,2025-03-20
105,Omar,Sales,71000,2025-04-05
```

2. Log in to Snowflake and create a quick table

1. Go to <https://app.snowflake.com> and log in

2. Left sidebar → Worksheets → + Worksheet

3. Copy-paste and run this once (this creates a fresh table for today's lab):

```
CREATE OR REPLACE TABLE EMPLOYEES (
  EMPLOYEE_ID INT,
  FIRST_NAME STRING,
  DEPARTMENT STRING,
  SALARY NUMBER,
  START_DATE DATE
);
```

3. Open the table in the UI

- Left sidebar → Databases

- Expand your account → SNOWFLAKE_SAMPLE_DATA is not needed – instead:

→ Click + Database at the top if you don't see one → name it CSV_LAB → Create

→ Expand CSV_LAB → PUBLIC → Tables

→ Click the EMPLOYEES table you just created

4. Start the Load Data wizard

Top-right corner → big blue button Load Data

5. Follow the wizard

Step	What You See on the Screen	What to Do	Click This Button
1. Warehouse	"Select a warehouse to run the load"	Choose your warehouse (usually COMPUTE_WH – it's already selected in trials)	Next
2. Select files	Big empty area with "Drag and drop files here" or a + button	Click the + button OR drag & drop your CSV file from the desktop	Next
3. File format	Dropdowns and checkboxes	• Set Format to CSV • Check the box First row contains column names • Leave everything else as default	Next
4. Load options	Error handling and load behaviour	• On error → Continue • Action → Append (so existing rows stay safe)	Load

6. Success

You will see a green message instantly:

Load completed successfully – 5 rows loaded

7. View your data (no SQL needed!)

Stay on the same screen → click the Data Preview tab (or refresh)

8. Bonus: Quick SQL check (optional)

Go back to your worksheet and run:

```
SELECT * FROM EMPLOYEES ORDER BY EMPLOYEE_ID;
```

Clean-up (optional)

```
DROP DATABASE IF EXISTS CSV_LAB;
```

Snowflake Compute

Step-by-Step

Step 1 – See the Warehouse You Are Currently Using

1. Look at the top-right corner of Snowflake

You will see something like: COMPUTE_WH ▼ (XS)

This is your current warehouse (the “engine” running your queries).

2. In a worksheet, run this :

```
SELECT CURRENT_WAREHOUSE(), CURRENT_USER();
```

You will see COMPUTE_WH in the results.

Step 2 – Create Your First Own Warehouse

1. Left sidebar → Admin → Warehouses

2. Top-right → blue button + Warehouse

Fill the form exactly like this :

Field	What to choose/type
Name	MY_TINY_WH
Size	X-Small
Auto Suspend	60 seconds (type 60)
Auto Resume	☒ Checked (important!)
Comment	My first warehouse – created in lab

Click Create Warehouse

You will see a green message: “Warehouse MY_TINY_WH successfully created”

Step 3 – Create a Second Bigger Warehouse

Repeat the same steps, but use these values:

Field	Value
Name	MY_BIG_WH
Size	Small
Auto Suspend	60 seconds
Auto Resume	☒ Checked
Comment	My bigger warehouse for faster queries

Click Create Warehouse

Step 4 – Switch Warehouse from the Top Bar

1. Top-right corner → click the dropdown next to your current warehouse
2. Choose MY_TINY_WH

The bar instantly changes to: MY_TINY_WH ▼ (XS)

Run this again in any worksheet:

```
SELECT CURRENT_WAREHOUSE();
```

Result: MY_TINY_WH

Step 5 Resize:

1. Go back to Admin → Warehouses
2. Find MY_BIG_WH → click the three dots ⋮ → Edit
3. Change Size from Small → Large
4. Click Save

Step 6 – Run a Query on Tiny vs Big Warehouse

We will run a simple “fake big” query to feel the difference.

In a worksheet, run this exact query twice:

This generates ~100 million rows (but runs fast on Snowflake)

```
SELECT SEQ4() AS N, RAND() AS R  
FROM TABLE(GENERATOR(ROWCOUNT => 100000000));
```

First time (while MY_TINY_WH is selected):

Now switch warehouse (top-right dropdown) → MY_BIG_WH (Large)

Run the exact same query again:

Step 7 – Suspend a Warehouse to Stop Credits

1. Go to Admin → Warehouses
2. Find MY_TINY_WH → click three dots ⋮ → Suspend

Status changes to “Suspended” and credits stop burning.

Do nothing for 60 seconds → both warehouses auto-suspend (because we set Auto Suspend = 60)

Step 8 – Resume Automatically

Just run any query again (e.g., SELECT 1;).

Watch the top bar: the warehouse automatically wakes up in 2–3 seconds and runs your query.

Summary – What You Learned

Create warehouse	+ Warehouse → filled form
Switch warehouse	Top-right dropdown
Resize (scale up/down)	Edit → change size → Save (seconds, no downtime)
Suspend / Auto-suspend	Suspend button or wait 60 seconds
Auto-resume	Run any query → warehouse wakes up automatically
Pay only for compute used	Credits stop when suspended

Implementing ETL and ELT

ShopNow, an e-commerce startup, receives daily CSVs from its website:

- `customers.csv`, `products.csv`, `orders.csv`, `order_items.csv`.

Data is messy (extra spaces, mixed date formats, missing fields). The analytics team needs a clean star schema in MySQL for dashboards and finance checks.

Goal: build two ingestion paths:

1. **ETL:** clean/transform **during** load → land directly into “clean” staging.
2. **ELT:** load **raw as-is**, then use SQL to transform into a star schema.

Objectives

- Create **raw** and **clean / warehouse** schemas.
- Demonstrate **ETL** using `LOAD DATA [LOCAL] INFILE ... SET ....`
- Demonstrate **ELT** using `INSERT ... SELECT` transformations from raw tables.
- Build a simple star schema: `dim_customer`, `dim_product`, `fact_order`.
- Verify with **row counts**, **referential integrity checks**, and **revenue totals**.

0) Prerequisites (one-time)

MySQL 8.x. For file loads you can use either:

- Server-side path within `secure_file_priv`, or
- Client-side with `LOCAL INFILE`.

Enable local infile if needed:

```
-- As admin:  
SET GLOBAL local_infile = 1;
```

```
-- When connecting with mysql client:  
-- mysql --local-infile=1 -u <user> -p
```

1) Sample CSV files

Use these minimal examples (expand later as you wish).

customers.csv

```
customer_id,full_name,email,country,created_at
1, Alice Carter ,alice@example.com,IN,2024-11-01
2,Bob Singh ,bob@example.com,IN,11/02/2024
3, ,charlie@example.com,US,2024-11-03
```

products.csv

```
product_id,sku,product_name,category,unit_price
10,S-1001,"USB-C Cable 1m",Accessories,6.99
11,S-1002,"USB-C Cable 2m",Accessories,7.99
12,S-2001,"Wireless Mouse",Peripherals,22.50
```

orders.csv

```
order_id,customer_id,order_date,currency,order_status
1001,1,2024-11-10,INR,PAID
1002,2,11/12/2024,INR,PAID
1003,3,2024-11-13,USD,CANCELLED
```

order_items.csv

```
order_item_id,order_id,product_id,qty,unit_price
5001,1001,10,2,6.99
5002,1001,11,1,7.99
5003,1002,12,1,22.50
5004,1003,11,2,7.99
```

Place these files under a path you can reference (e.g., your home dir). If you use server-side loads, copy them into the MySQL server's `secure_file_priv` folder.

2) Create databases & users

```
-- As an admin or a user with create privileges
CREATE DATABASE IF NOT EXISTS shopnow_raw;
CREATE DATABASE IF NOT EXISTS shopnow_dw;

-- (Optional) create a dedicated user and grant:
-- CREATE USER 'etl_user'@'%' IDENTIFIED BY 'StrongP@ssw0rd';
-- GRANT ALL ON shopnow_raw.* TO 'etl_user'@'%';
-- GRANT ALL ON shopnow_dw.* TO 'etl_user'@'%';
-- FLUSH PRIVILEGES;
```

3) ELT path – “Load first, then transform”

3.1 Create **raw** tables (all text-friendly to accept messy data)

```
USE shopnow_raw;
```

```
DROP TABLE IF EXISTS customers_raw;  
CREATE TABLE customers_raw (  
  customer_id VARCHAR(50),  
  full_name   VARCHAR(255),  
  email       VARCHAR(255),  
  country     VARCHAR(50),  
  created_at  VARCHAR(50)  
);
```

```
DROP TABLE IF EXISTS products_raw;  
CREATE TABLE products_raw (  
  product_id VARCHAR(50),  
  sku         VARCHAR(50),  
  product_name VARCHAR(255),  
  category    VARCHAR(100),  
  unit_price  VARCHAR(50)  
);
```

```
DROP TABLE IF EXISTS orders_raw;  
CREATE TABLE orders_raw (  
  order_id   VARCHAR(50),  
  customer_id VARCHAR(50),  
  order_date VARCHAR(50),  
  currency   VARCHAR(10),  
  order_status VARCHAR(50)  
);
```

```
DROP TABLE IF EXISTS order_items_raw;  
CREATE TABLE order_items_raw (  
  order_item_id VARCHAR(50),  
  order_id       VARCHAR(50),  
  product_id     VARCHAR(50),  
  qty            VARCHAR(50),  
  unit_price     VARCHAR(50)  
);
```

3.2 Load raw CSVs (ELT: no transformation yet)

Use **one** of the two forms below:

- **LOCAL** (client-side): replace `/path/...` with your actual path
- **NON-LOCAL**: put files inside `secure_file_priv` and drop **LOCAL**


```
LOAD DATA LOCAL INFILE '/path/customers.csv'  
INTO TABLE customers_raw  
FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1 LINES;
```

```
LOAD DATA LOCAL INFILE '/path/products.csv'  
INTO TABLE products_raw  
FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1 LINES;
```

```
LOAD DATA LOCAL INFILE '/path/orders.csv'  
INTO TABLE orders_raw  
FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1 LINES;
```

```
LOAD DATA LOCAL INFILE '/path/order_items.csv'  
INTO TABLE order_items_raw  
FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1 LINES;
```

3.3 Transform & load into **DW star schema** (ELT)

Create **dimensions & fact**:

```
USE shopnow_dw;
```

```
-- Dimensions  
DROP TABLE IF EXISTS dim_customer;  
CREATE TABLE dim_customer (  
  customer_key INT AUTO INCREMENT PRIMARY KEY,  
  customer_id INT UNIQUE,  
  full_name VARCHAR(255),  
  email VARCHAR(255),  
  country VARCHAR(50),  
  created_date DATE  
);
```

```
DROP TABLE IF EXISTS dim_product;  
CREATE TABLE dim_product (  
  product_key INT AUTO INCREMENT PRIMARY KEY,  
  product_id INT UNIQUE,  
  sku VARCHAR(50),  
  product_name VARCHAR(255),  
  category VARCHAR(100),  
  unit_price DECIMAL(10,2)  
);
```

```
-- Fact table  
DROP TABLE IF EXISTS fact_order;  
CREATE TABLE fact_order (  
  order_key INT AUTO INCREMENT PRIMARY KEY,  
  order_id INT,  
  customer_key INT,  
  order_date DATE,
```

```

currency CHAR(3),
order_status VARCHAR(50),
product_key INT,
qty INT,
item_price DECIMAL(10,2),
line_amount DECIMAL(12,2),
CONSTRAINT fk_fact_cust FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key),
CONSTRAINT fk_fact_prod FOREIGN KEY (product_key) REFERENCES dim_product(product_key)
);

```

ELT Transformation SQL:

- Clean spaces, standardize dates (accepting YYYY-MM-DD or MM/DD/YYYY), cast types, and drop cancelled orders.

```

-- 1) Load dim_customer from raw (ELT)
INSERT INTO dim_customer (customer_id, full_name, email, country, created_date)
SELECT
  CAST(NULLIF(TRIM(customer_id), '') AS SIGNED),
  NULLIF(TRIM(full_name), ''),
  NULLIF(TRIM(email), ''),
  UPPER(NULLIF(TRIM(country), '')),
  -- try ISO first, else US format
  COALESCE(
    STR_TO_DATE(NULLIF(TRIM(created_at), ''), '%Y-%m-%d'),
    STR_TO_DATE(NULLIF(TRIM(created_at), ''), '%m/%d/%Y')
  ) AS created_date
FROM shopnow_raw.customers_raw
WHERE NULLIF(TRIM(email), '') IS NOT NULL; -- simple business rule

```

```

-- 2) Load dim_product from raw (ELT)
INSERT INTO dim_product (product_id, sku, product_name, category, unit_price)
SELECT
  CAST(NULLIF(TRIM(product_id), '') AS SIGNED),
  NULLIF(TRIM(sku), ''),
  NULLIF(TRIM(product_name), ''),
  NULLIF(TRIM(category), ''),
  CAST(NULLIF(TRIM(unit_price), '') AS DECIMAL(10,2))
FROM shopnow_raw.products_raw;

```

```

-- 3) Build a helper view for valid orders
DROP VIEW IF EXISTS v_orders_clean;
CREATE VIEW v_orders_clean AS
SELECT
  CAST(NULLIF(TRIM(o.order_id), '') AS SIGNED) AS order_id,
  CAST(NULLIF(TRIM(o.customer_id), '') AS SIGNED) AS customer_id,
  COALESCE(
    STR_TO_DATE(NULLIF(TRIM(o.order_date), ''), '%Y-%m-%d'),
    STR_TO_DATE(NULLIF(TRIM(o.order_date), ''), '%m/%d/%Y')
  ) AS order_date,

```

```

    UPPER(NULLIF(TRIM(o.currency), ""))          AS currency,
    UPPER(NULLIF(TRIM(o.order_status), ""))       AS order_status
FROM shopnow_raw.orders_raw o;

-- 4) Load fact_order from raw order items + cleaned orders + dims
INSERT INTO fact_order (order_id, customer_key, order_date, currency, order_status,
                       product_key, qty, item_price, line_amount)
SELECT
    o.order_id,
    dc.customer_key,
    o.order_date,
    o.currency,
    o.order_status,
    dp.product_key,
    CAST(NULLIF(TRIM(oi.qty), "") AS SIGNED)          AS qty,
    CAST(NULLIF(TRIM(oi.unit_price), "") AS DECIMAL(10,2)) AS item_price,
    CAST(NULLIF(TRIM(oi.qty), "") AS SIGNED) *
    CAST(NULLIF(TRIM(oi.unit_price), "") AS DECIMAL(10,2)) AS line_amount
FROM shopnow_raw.order_items_raw oi
JOIN v_orders_clean o
  ON o.order_id = CAST(NULLIF(TRIM(oi.order_id), "") AS SIGNED)
JOIN dim_customer dc
  ON dc.customer_id = o.customer_id
JOIN dim_product dp
  ON dp.product_id = CAST(NULLIF(TRIM(oi.product_id), "") AS SIGNED)
WHERE o.order_status = 'PAID'; -- business rule: exclude cancelled

```

4) ETL path – “Transform during load”

Here we load the CSV **directly** into a “clean staging” area using **LOAD DATA** with **SET** transformations (trim, cast, date parsing). This is **ETL** because transforms happen *before/while loading*.

4.1 Create **clean staging** tables

```

USE shopnow_dw;

DROP TABLE IF EXISTS stg_customers_etl;
CREATE TABLE stg_customers_etl (
    customer_id INT,
    full_name VARCHAR(255),
    email VARCHAR(255),
    country CHAR(2),
    created_date DATE
);

DROP TABLE IF EXISTS stg_order_items_etl;

```

```
CREATE TABLE stg_order_items_etl (
  order_id INT,
  product_id INT,
  qty INT,
  unit_price DECIMAL(10,2)
);
```

4.2 Load with **on-the-fly transforms** (ETL)

```
-- ETL for customers: trim, uppercase country, parse date variants
LOAD DATA LOCAL INFILE '/path/customers.csv'
INTO TABLE stg_customers_etl
FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1 LINES
(@customer_id,@full_name,@email,@country,@created_at)
SET
  customer_id = CAST(NULLIF(TRIM(@customer_id),"") AS SIGNED),
  full_name = NULLIF(TRIM(@full_name),""),
  email = NULLIF(TRIM(@email),""),
  country = UPPER(SUBSTRING(NULLIF(TRIM(@country),""),1,2)),
  created_date = COALESCE(
    STR_TO_DATE(NULLIF(TRIM(@created_at),""), '%Y-%m-%d'),
    STR_TO_DATE(NULLIF(TRIM(@created_at),""), '%m/%d/%Y')
  );
```

```
-- ETL for order_items: trim + cast
LOAD DATA LOCAL INFILE '/path/order_items.csv'
INTO TABLE stg_order_items_etl
FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1 LINES
(@order_item_id,@order_id,@product_id,@qty,@unit_price)
SET
  order_id = CAST(NULLIF(TRIM(@order_id),"") AS SIGNED),
  product_id = CAST(NULLIF(TRIM(@product_id),"") AS SIGNED),
  qty = CAST(NULLIF(TRIM(@qty),"") AS SIGNED),
  unit_price = CAST(NULLIF(TRIM(@unit_price),"") AS DECIMAL(10,2));
```

In a pure ETL pipeline you would apply **all** business rules here and load straight into **dim_*** / **fact_***. For clarity we just showed two “during-load” transforms to contrast with the ELT approach above.

5) Indexes & helpful constraints

```
USE shopnow_dw;
CREATE INDEX ix_dim_customer_id ON dim_customer(customer_id);
CREATE INDEX ix_dim_product_id ON dim_product(product_id);

CREATE INDEX ix_fact_order_date ON fact_order(order_date);
CREATE INDEX ix_fact_order_customer ON fact_order(customer_key);
CREATE INDEX ix_fact_order_product ON fact_order(product_key);
```

6) Verification (run after ELT + ETL steps)

6.1 Row counts & sanity

-- Raw counts

```
SELECT 'customers_raw' AS t, COUNT(*) c FROM shopnow_raw.customers_raw
UNION ALL SELECT 'products_raw', COUNT(*) FROM shopnow_raw.products_raw
UNION ALL SELECT 'orders_raw', COUNT(*) FROM shopnow_raw.orders_raw
UNION ALL SELECT 'order_items_raw', COUNT(*) FROM shopnow_raw.order_items_raw;
```

-- DW counts

```
SELECT 'dim_customer' AS t, COUNT(*) c FROM shopnow_dw.dim_customer
UNION ALL SELECT 'dim_product', COUNT(*) FROM shopnow_dw.dim_product
UNION ALL SELECT 'fact_order', COUNT(*) FROM shopnow_dw.fact_order;
```

-- ETL staging counts (optional)

```
SELECT 'stg_customers_etl' AS t, COUNT(*) c FROM shopnow_dw.stg_customers_etl
UNION ALL SELECT 'stg_order_items_etl', COUNT(*) FROM shopnow_dw.stg_order_items_etl;
```

6.2 Referential integrity checks

-- Any fact rows referencing missing customers?

```
SELECT fo.order_id
FROM shopnow_dw.fact_order fo
LEFT JOIN shopnow_dw.dim_customer dc ON fo.customer_key = dc.customer_key
WHERE dc.customer_key IS NULL;
```

-- Any fact rows referencing missing products?

```
SELECT fo.order_id
FROM shopnow_dw.fact_order fo
LEFT JOIN shopnow_dw.dim_product dp ON fo.product_key = dp.product_key
WHERE dp.product_key IS NULL;
```

6.3 Business rule checks

-- Cancelled orders should NOT appear in fact_order

```
SELECT DISTINCT order_id, order_status
FROM shopnow_dw.fact_order
WHERE order_status <> 'PAID';
```

-- Revenue totals (should equal sum(qty * unit_price) for PAID orders only)

```
SELECT
    SUM(line_amount) AS total_revenue
FROM shopnow_dw.fact_order;
```

```
-- Cross-check revenue per order
SELECT order_id, SUM(line_amount) AS order_revenue
FROM shopnow_dw.fact_order
GROUP BY order_id
ORDER BY order_id;
```

6.4 Sample query for the business

```
-- Top products by revenue
SELECT
  dp.product_name,
  SUM(fo.line_amount) AS revenue
FROM shopnow_dw.fact_order fo
JOIN shopnow_dw.dim_product dp ON fo.product_key = dp.product_key
GROUP BY dp.product_name
ORDER BY revenue DESC;
```

7) Incremental loads

Upserts for dims (idempotent):

```
-- Example: product dimension upsert from raw
INSERT INTO shopnow_dw.dim_product (product_id, sku, product_name, category, unit_price)
SELECT CAST(product_id AS SIGNED), sku, product_name, category, CAST(unit_price AS DECIMAL(10,2))
FROM shopnow_raw.products_raw
ON DUPLICATE KEY UPDATE
  sku = VALUES(sku),
  product_name = VALUES(product_name),
  category = VALUES(category),
  unit_price = VALUES(unit_price);
```

Late-arriving facts: insert facts only if related dims exist; otherwise stage and reprocess later.

8) What's ETL vs ELT here?

- **ETL** section used `LOAD DATA ... SET` to **transform** as rows were loaded into `stg_*` (and could go straight to dims/facts).
- **ELT** section landed the messy CSVs into `*_raw` tables first, then used **SQL** transformations (`INSERT ... SELECT` + casting + date normalization) to populate `dim_*` and `fact_*`.

Both approaches are valid; organizations often use a hybrid: land raw, then prefer ELT for transparency and repeatability, using ETL for heavy cleansing when needed.